Swift Coroutines with KMP-NativeCoroutines

Aleksander Jaworski @ FootballCo - 07.07.2023 Droidcon Berlin

Aleksander Jaworski

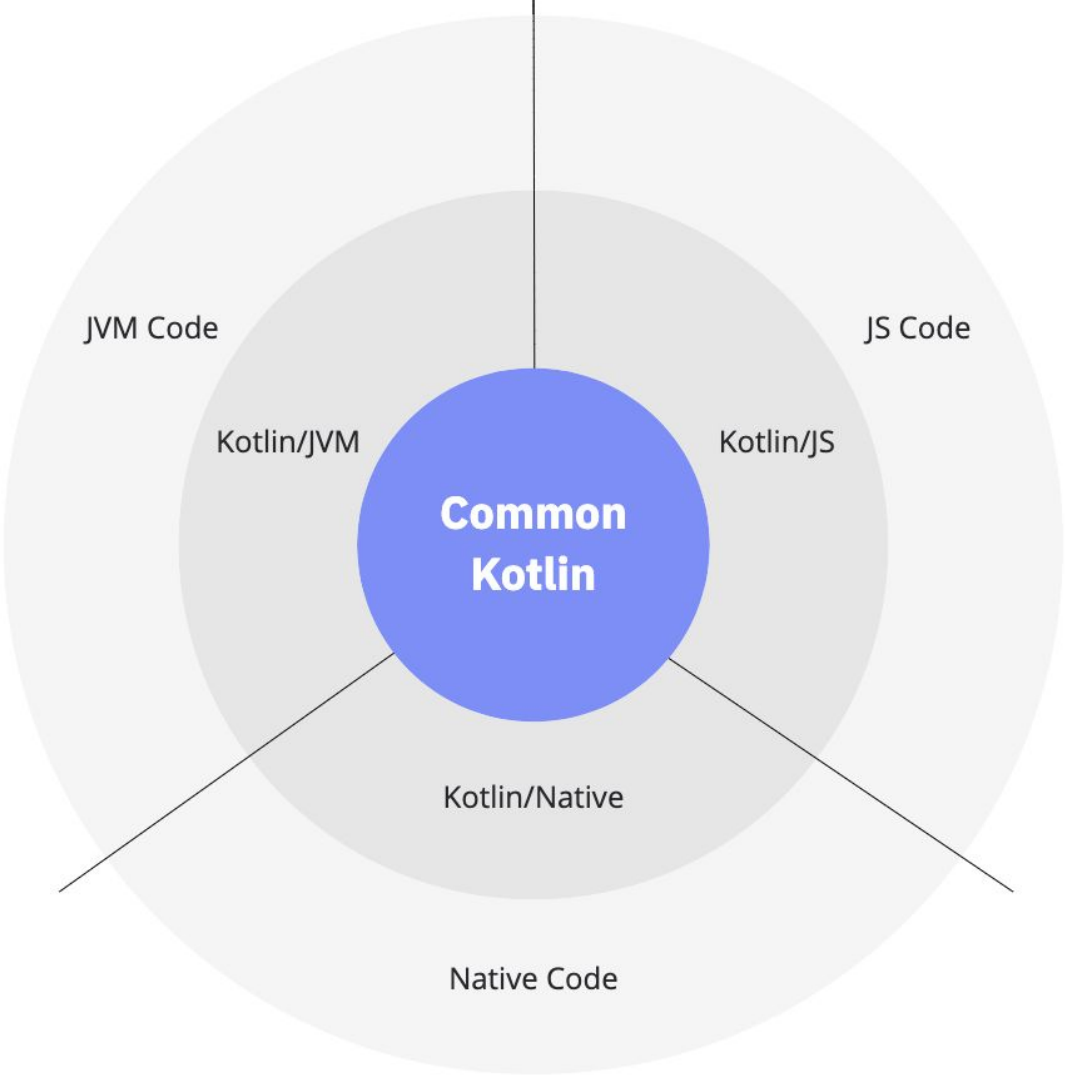
- Android Developer @ FootballCo
- Blog **akjaw.com** (Kotlin Multiplatform and Testing)
- Twitter @akjaworski1



What is Kotlin Multiplatform?

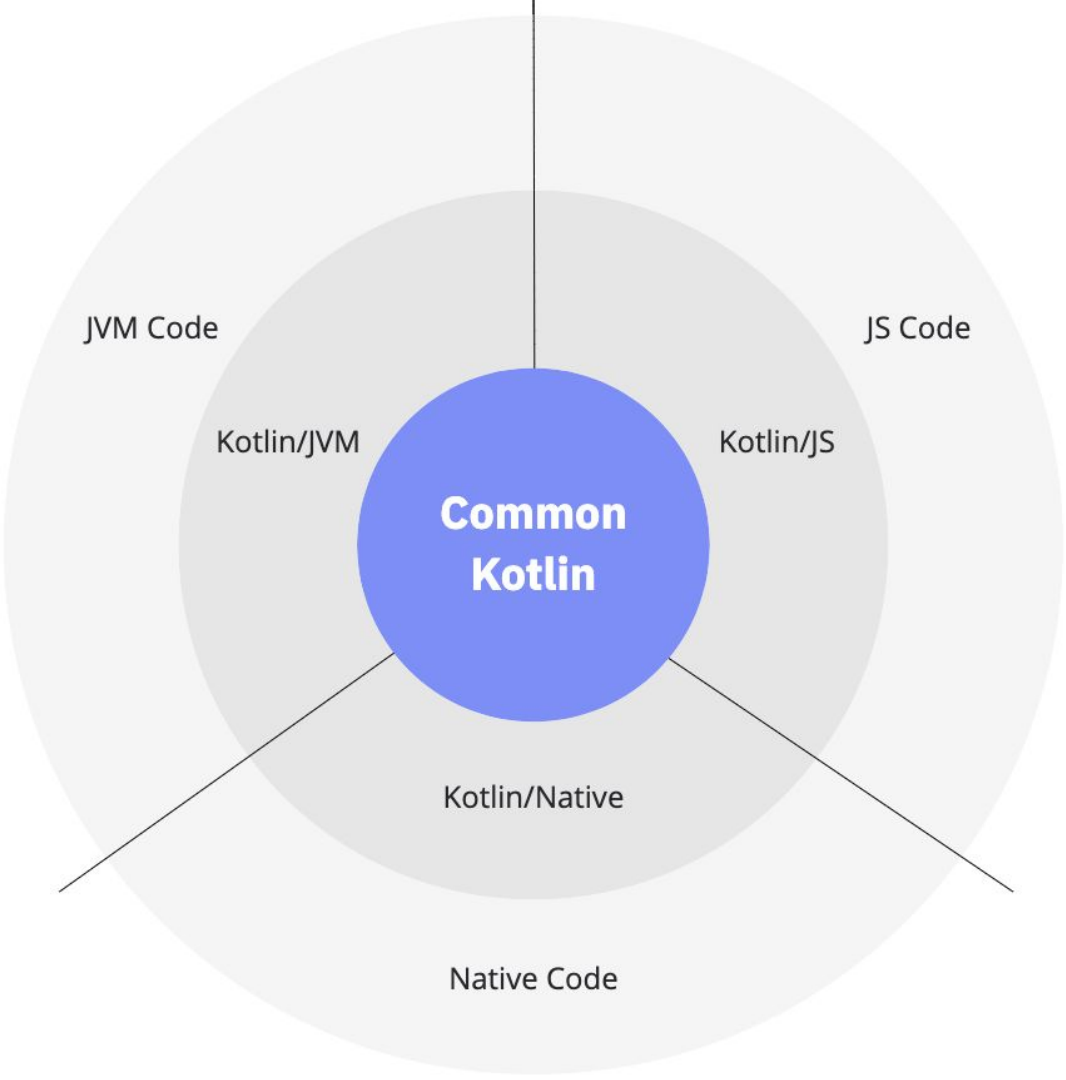
What is Kotlin Multiplatform?

- One codebase, different platforms (iOS, Desktop, Web)



What is Kotlin Multiplatform?

- One codebase, different platforms (iOS, Desktop, Web)
- Kotlin has the logic,platform has the UI*



* Compose Multiplatform is a thing

How to call Kotlin from Swift?

- Just call the generated Kotlin Multiplatform framework

How to call Kotlin from Swift?

- Just call the generated Kotlin Multiplatform framework

```
// Kotlin
fun execute() {
   print("Exe")
}
```

How to call Kotlin from Swift?

- Just call the generated Kotlin Multiplatform framework

```
// Kotlin
fun execute() {
   print("Exe")
}
```

```
// Generated
open func execute()
```

How to call Kotlin from Swift?

- Just call the generated Kotlin Multiplatform framework

```kotlin
// Kotlin
fun execute() {
    print("Exe")
}
```

```
// Generated
open func execute()
```

```
// Swift
execute()
```

How to call coroutines from Swift?

How to call coroutines from Swift?

- It’s not so simple...

How to call coroutines from Swift?

- It’s not so simple...

```
// Kotlin
suspend fun execute() {
   print("44af1d53beeb")
}
```

How to call coroutines from Swift?

- It’s not so simple...

```
// Kotlin
suspend fun execute() {
   print("44af1d53beeb")
}
```

```
// Generated
 * @note This method converts instances of
CancellationException to errors.
 * Other uncaught Kotlin exceptions are
fatal.
open func execute(completionHandler:

@escaping (Error?) -> Void)
```

How to call coroutines from Swift?

- It’s not so simple...

```
// Kotlin
suspend fun execute() {
   print("44af1d53beeb")
}
```

```
// Generated
 * @note This method converts instances of
CancellationException to errors.
 * Other uncaught Kotlin exceptions are
fatal.
open func execute(completionHandler:

@escaping (Error?) -> Void)
```

```
// Swift
execute { error in
    print(error)
}
```

How to call coroutines from Swift?

- It’s not so simple...

```
// Kotlin
suspend fun execute() {
   print("44af1d53beeb")
}
```

- How to cancel?

```
// Generated
 * @note This method converts instances of
CancellationException to errors.
 * Other uncaught Kotlin exceptions are
fatal.
open func execute(completionHandler:

@escaping (Error?) -> Void)
```

```
// Swift
execute { error in
    print(error)
}
```

How to call coroutines from Swift?

- It’s not so simple...

```
// Kotlin
suspend fun execute() {
   print("44af1d53beeb")
}
```

- How to cancel?
- Changing Threads?

```
// Generated
 * @note This method converts instances of
CancellationException to errors.
 * Other uncaught Kotlin exceptions are
fatal.
open func execute(completionHandler:

@escaping (Error?) -> Void)
```

```
// Swift
execute { error in
    print(error)
}
```

How to call coroutines from Swift?

- It’s not so simple...

```kotlin
// Kotlin
suspend fun execute() {
    print("44af1d53beeb")
}
```

- How to cancel?
- Changing Threads?
- Exception handling?

```
// Generated
 * @note This method converts instances of
CancellationException to errors.
 * Other uncaught Kotlin exceptions are
fatal.
open func execute(completionHandler:
@escaping (Error?) -> Void)
```

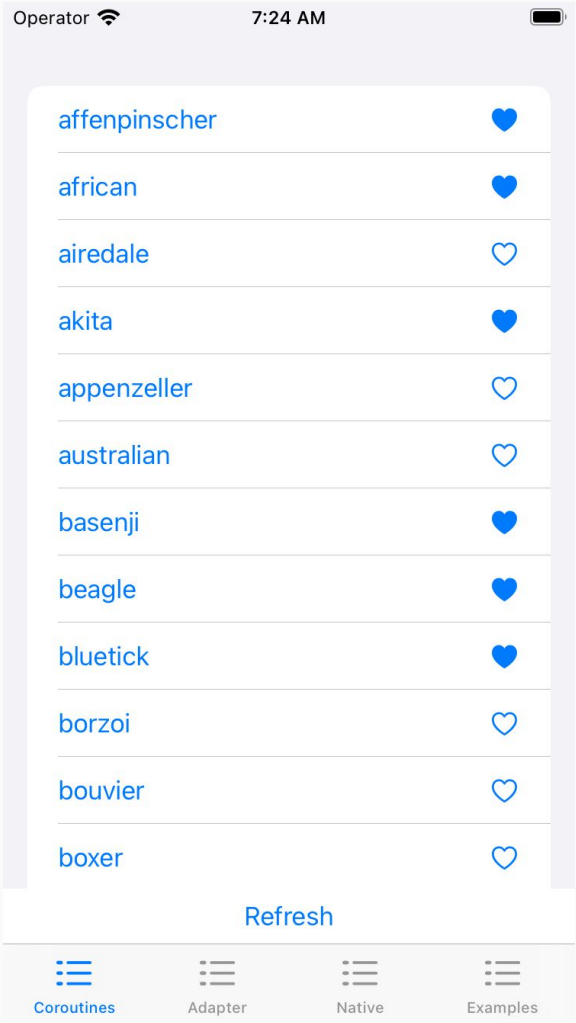
```
// Swift
execute { error in
    print(error)
}
```

Agenda

- Calling default Suspend and Flows directly from Swift
- Creating Coroutine Adapters
- Using KMP-NativeCoroutines
- SwiftUI integrations
- Exception Handling
- Cancellation

The App

- KaMPKit Starter from Touchlab
- Downloads from API i and saves to a Database
- 3 Coroutines call implementations



The App

- KaMPKit Starter from Touchlab
- Downloads from API i and saves to a Database
- 3 Coroutines call implementations



github.com/AKJAW/Swift-Coroutines-With-KMP-NativeCoroutines

What are we calling

- Ordinary function
- Suspend function
- Data stream

```
class BreedViewModel() : ViewModel() {

    val breedState: StateFlow<BreedViewState>

    suspend fun refreshBreeds(): Boolean

    fun updateBreedFavorite(breed: Breed): Job
}
```

What are we calling

- Ordinary function
- Suspend function
- Data stream

```
class BreedViewModel() : ViewModel() {

    val breedState: StateFlow<BreedViewState>

    suspend fun refreshBreeds(): Boolean

    fun updateBreedFavorite(breed: Breed): Job
}


actual abstract class ViewModel {

    actual val viewModelScope = MainScope()

    fun clear() {
        viewModelScope.coroutineContext
         .cancelChildren()
    }
}
```

The easiest one is the Ordinary function




```
fun updateBreedFavorite(breed: Breed): Job {
    return viewModelScope.launch {
        breedRepository.updateBreedFavorite(breed)
    }
}
```

The easiest one is the Ordinary function




```
fun updateBreedFavorite(breed: Breed): Job {
    return viewModelScope.launch {
        breedRepository.updateBreedFavorite(breed)
    }
}
```

```
func onBreedFavorite(_ breed: Breed) {
    viewModel?.updateBreedFavorite(breed: breed)
}
```

Ordinary functions

- Fire and forget
- Modify internal state
- CoroutineScope is available

```
fun updateBreedFavorite(breed: Breed): Job {
    return viewModelScope.launch {
        breedRepository.updateBreedFavorite(breed)
    }
}
```

```
func onBreedFavorite(_ breed: Breed) {
    viewModel?.updateBreedFavorite(breed: breed)
}
```

Suspend functions

Suspending functions in Swift

- Default generated framework

Suspending functions in Swift

- Default generated framework
- Manual Adaptera / Wrappera

Suspending functions in Swift

- Default generated framework
- Manual Adaptera / Wrappera
- KMP-NativeCoroutines Plugin

Suspending functions

- We want to know when it completes
- The return value is needed
- We want the ability to cancel it

Suspending functions

- We want to know when it completes
- The return value is needed
- We want the ability to cancel it

```
suspend fun refreshBreeds(): Boolean {
    return try {
        breedRepository.refreshBreeds()
        true
    } catch (exception: Exception) {
        handleBreedError(exception)
        false
    }
}
```

Generated function

- Cannot be cancelled
- Forces an Optional value

```
func refresh() {
  viewModel?.refreshBreeds { wasRefreshed, error in
    print("\(wasRefreshed), \(error)")
  }
}
```

Generated function

- Cannot be cancelled
- Forces an Optional value
- **No error handling**

```
func refresh() {
  viewModel?.refreshBreeds { wasRefreshed, error in
    print("\(wasRefreshed), \(error)")
  }
}
```

```
  /**
   * @note This method converts instances of
  CancellationException to errors.
   * Other uncaught Kotlin exceptions are fatal.
  */
  - (void)refreshBreedsWithCompletionHandler:(void
  (^)(SharedBoolean * _Nullable, NSError *
  _Nullable))completionHandler
  __attribute__((swift_name("refreshBreeds(completionHandler:)"))
  );
```

Adapter / Wrapper - Kotlin

```
class SuspendAdapter<T : Any>(
    private val scope: CoroutineScope,
    private val suspender: suspend () -> T
) {

  fun subscribe(
      onSuccess: (item: T) -> Unit,
      onThrow: (error: Throwable) -> Unit
  ) = scope.launch {
      try {
          onSuccess(suspender())
      } catch (error: Throwable) {
          onThrow(error)
      }
  }

}
```

Adapter / Wrapper - Kotlin

```
class SuspendAdapter<T : Any>(
    private val scope: CoroutineScope,
    private val suspender: suspend () -> T
) {

  fun subscribe(
      onSuccess: (item: T) -> Unit,
      onThrow: (error: Throwable) -> Unit
  ) = scope.launch {
      try {
          onSuccess(suspender())
      } catch (error: Throwable) {
          onThrow(error)
      }
  }

}
```

```
fun refreshBreeds() =
  SuspendAdapter(viewModel.scope) {
    viewModel.refreshBreeds()
  }
```

Adapter / Wrapper - Kotlin

```kotlin
class SuspendAdapter<T : Any>(
    private val scope: CoroutineScope,
    private val suspender: suspend () -> T
) {

 fun subscribe(
     onSuccess: (item: T) -> Unit,
     onThrow: (error: Throwable) -> Unit
 ) = scope.launch {
     try {
         onSuccess(suspender())
     } catch (error: Throwable) {
         onThrow(error)
     }
 }
}
```

```kotlin
fun refreshBreeds() =
  SuspendAdapter(viewModel.scope) {
    viewModel.refreshBreeds()
  }
```

Adapter/Wrapper - Swift

- Better error handling
- No Optional value
- Additional boilerplate

```
viewModel.refreshBreeds().subscribe(
    onSuccess: { value in
        print("completion \(value)")
    },
    onThrow: { error in
        print("error \(error)")
    }
)
```

Adapter/Wrapper - Swift

- Better error handling
- No Optional value
- Additional boilerplate
- Ability to integrate with Combine

```
viewModel.refreshBreeds().subscribe(
    onSuccess: { value in
        print("completion \(value)")
    },
    onThrow: { error in
        print("error \(error)")
    }
)
```

```
createFuture(suspendAdapter: adapter)
    .sink { completion in
        print("completion \(completion)")
    } receiveValue: { value in
        print("recieveValue \(value)")
    }.store(in: &cancellables)
```

KMP-NativeCoroutines

Suspend KMP-NativeCoroutines

```
@NativeCoroutines
suspend fun nativeRefreshBreeds(): Boolean =
    refreshBreeds()
```

NativeCoroutines

- No boilerplate

```
@NativeCoroutines
suspend fun nativeRefreshBreeds(): Boolean =
    refreshBreeds()
```

NativeCoroutines

- **No boilerplate**
- Error handling
- No Optional
- **Dedicated Swift packages** Combine, Async, RxSwift

```
@NativeCoroutines
suspend fun nativeRefreshBreeds(): Boolean =
    refreshBreeds()
```

```
let suspend = viewModel.nativeRefreshBreeds()
createFuture(for: suspend)
    .sink { completion in
        print("completion \(completion)")
    } receiveValue: { value in
        print("recieveValue \(value)")
    }
```

NativeCoroutines

- **No boilerplate**
- Error handling
- No Optional
- **Dedicated Swift packages** Combine, Async, RxSwift
- **Automatic cancelling**

```
@NativeCoroutines
suspend fun nativeRefreshBreeds(): Boolean =
    refreshBreeds()
```

```
let suspend = viewModel.nativeRefreshBreeds()
createFuture(for: suspend)
    .sink { completion in
        print("completion \(completion)")
    } receiveValue: { value in
        print("recieveValue \(value)")
    }.store(in: &cancellables)
```

Flow Data Streams

Data streams

- Generated framework - Many limitations + Boilerplate
- Adapter / Wrapper - Additional boilerplate
- KMP Native Coroutines

Flow Generated framework

```
class Collector<T>:
Kotlinx_coroutines_coreFlowCollector {
    private let callback: (T) -> Void

    init(callback: @escaping (T) -> Void) {
        self.callback = callback
    }

    func emit(value: Any?, completionHandler:
@escaping (Error?) -> Void) {
        callback(value as! T)
        completionHandler(nil)
    }
}
```

Flow Generated framework

```
class Collector<T>:
Kotlinx_coroutines_coreFlowCollector {
    private let callback: (T) -> Void

    init(callback: @escaping (T) -> Void) {
        self.callback = callback
    }

    func emit(value: Any?, completionHandler:
@escaping (Error?) -> Void) {
        callback(value as! T)
        completionHandler(nil)
    }
}
```

```
viewModel.breedState.collect(
    collector: Collector<BreedViewState> {
        [weak self] dogsState in

        self?.loading = dogsState.isLoading
        self?.breeds = dogsState.breeds
        self?.error = dogsState.error
    },
    completionHandler: { error in
        print("breed collection completion
error: \(error)")
    }
)
```

Flow Adapter / Wrapper

```
class FlowAdapter<T : Any>(
 private val scope: CoroutineScope,
 private val flow: Flow<T>
) {
 fun subscribe(
   onEach: (item: T) -> Unit,
   onComplete: () -> Unit,
   onThrow: (error: Throwable) -> Unit
 ): Canceller = JobCanceller(
   flow.onEach { onEach(it) }
     .catch { onThrow(it) }
     .onCompletion { onComplete() }
     .launchIn(scope)
 )
}

interface Canceller {
 fun cancel()
}

private class JobCanceller(private val job:
Job) : Canceller {
 override fun cancel() {
   job.cancel()
 }
}
```

Flow Adapter / Wrapper

```
class FlowAdapter<T : Any>(
 private val scope: CoroutineScope,
 private val flow: Flow<T>
) {
 fun subscribe(
   onEach: (item: T) -> Unit,
   onComplete: () -> Unit,
   onThrow: (error: Throwable) -> Unit
 ): Canceller = JobCanceller(
   flow.onEach { onEach(it) }
     .catch { onThrow(it) }
     .onCompletion { onComplete() }
     .launchIn(scope)
 )
}

interface Canceller {
 fun cancel()
}

private class JobCanceller(private val job:
Job) : Canceller {
 override fun cancel() {
   job.cancel()
 }
}
```

```
viewModel.breeds.subscribe(
    onEach: { [weak self] dogsState in
        self?.loading = dogsState.isLoading
        self?.breeds = dogsState.breeds
        self?.error = dogsState.error
    },
    onComplete: { print("Subscription end") },
    onThrow: { error in  print("Subscription
error: \(error)") }
)
```

Flow KMP-NativeCoroutines

```
private val mutableBreedState =
    MutableStateFlow(
        BreedViewState(isLoading = true)
    )
```

```
@NativeCoroutinesState
val nativeBreedState: StateFlow<BreedViewState> =
    breedState
```

Flow KMP-NativeCoroutines

```
private val mutableBreedState =
    MutableStateFlow(
        BreedViewState(isLoading = true)
    )

@NativeCoroutinesState
val nativeBreedState: StateFlow<BreedViewState> =
    breedState
```

Native Flow

- Integration with Native solutions

```
let nativeFlow = viewModel.nativeBreedStateFlow

createPublisher(for: nativeFlow)
```

Native Flow

- Integration with Native solutions

```
let nativeFlow = viewModel.nativeBreedStateFlow

createPublisher(for: nativeFlow)

    .sink { completion in
        print("Breeds completion \(completion)")
    } receiveValue: { [weak self] dogsState in
        self?.loading = dogsState.isLoading
        self?.breeds = dogsState.breeds
        self?.error = dogsState.error
    }
```

Native Flow

- Integration with Native solutions
- Collection on **Default dispatcher**

```
let nativeFlow = viewModel.nativeBreedStateFlow

createPublisher(for: nativeFlow)
    .receive(on: DispatchQueue.main)
    .sink { completion in
        print("Breeds completion \(completion)")
    } receiveValue: { [weak self] dogsState in
        self?.loading = dogsState.isLoading
        self?.breeds = dogsState.breeds
        self?.error = dogsState.error
    }
```

Native Flow

- Integration with Native solutions
- Collection on Default dispatcher
- **Automatic cancellation**

```
let nativeFlow = viewModel.nativeBreedStateFlow

createPublisher(for: nativeFlow)
    .receive(on: DispatchQueue.main)
    .sink { completion in
        print("Breeds completion \(completion)")
    } receiveValue: { [weak self] dogsState in
        self?.loading = dogsState.isLoading
        self?.breeds = dogsState.breeds
        self?.error = dogsState.error
    }
    .store(in: &cancellables)
```

Controlling the Scope

- Ability to cancel directly from Kotlin
- Specify which thread should the suspension happen

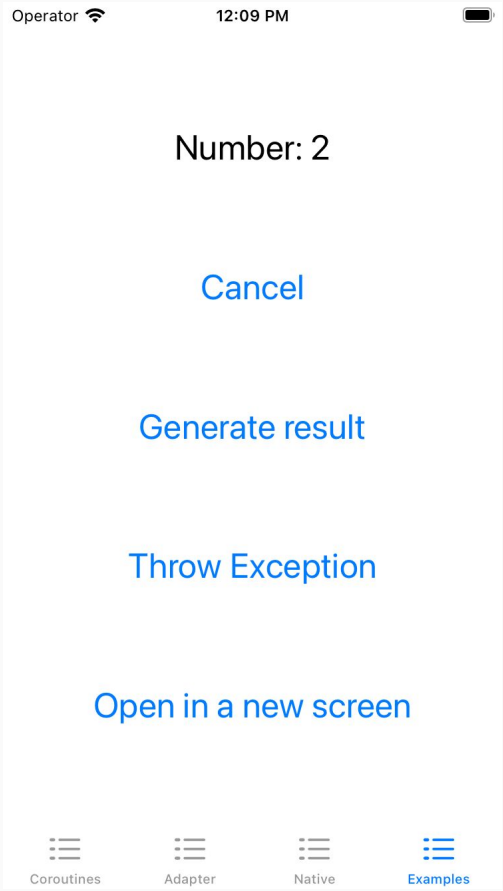
```
@NativeCoroutineScope
actual val viewModelScope =
    MainScope()
```

SwiftUI

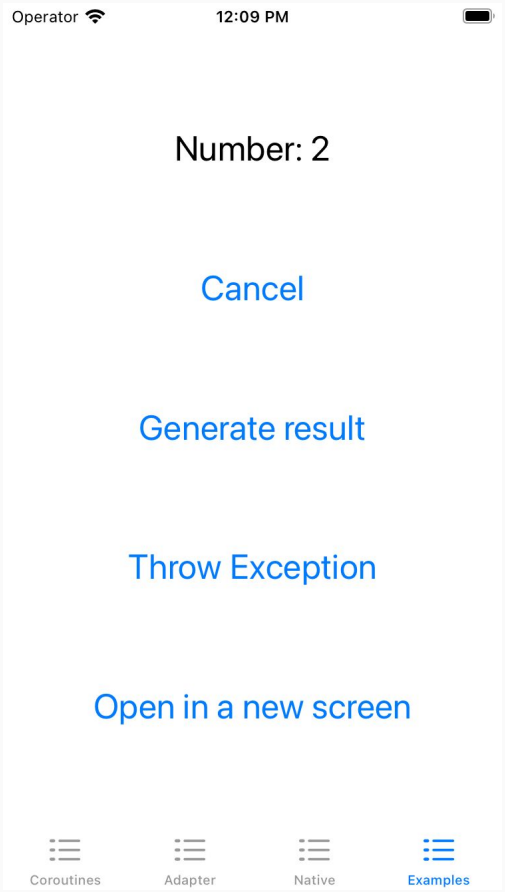
SwiftUI Integration

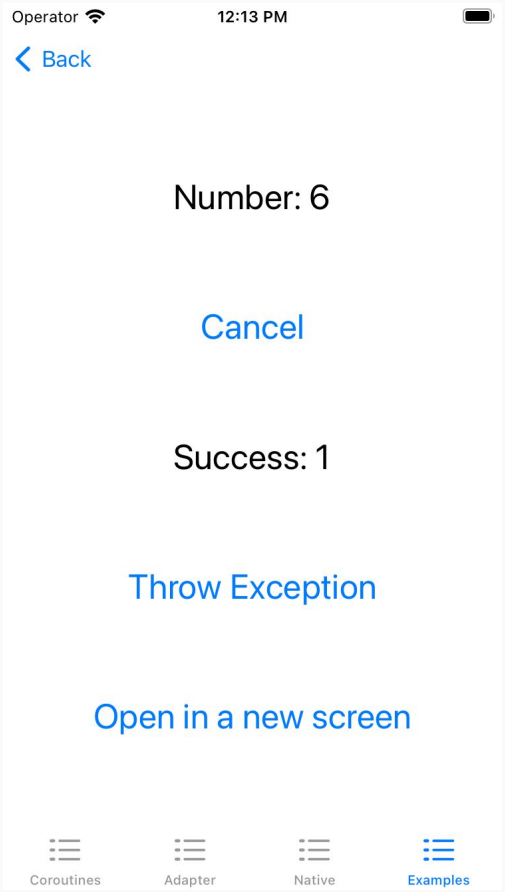
- Combine
- ObservableObject
- @Published
- @StateObject / @ObservedObject

Playground

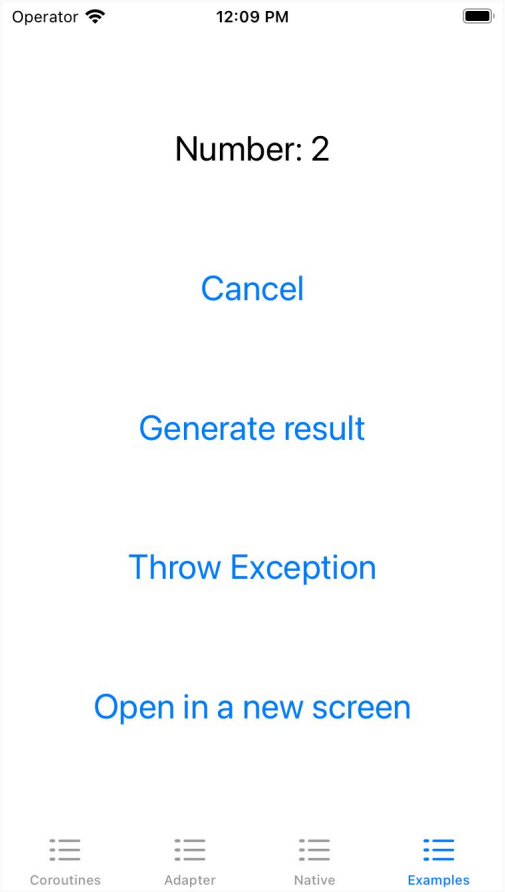

Operator
12:09 PM
Number: 2
Cancel
Generate result
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples

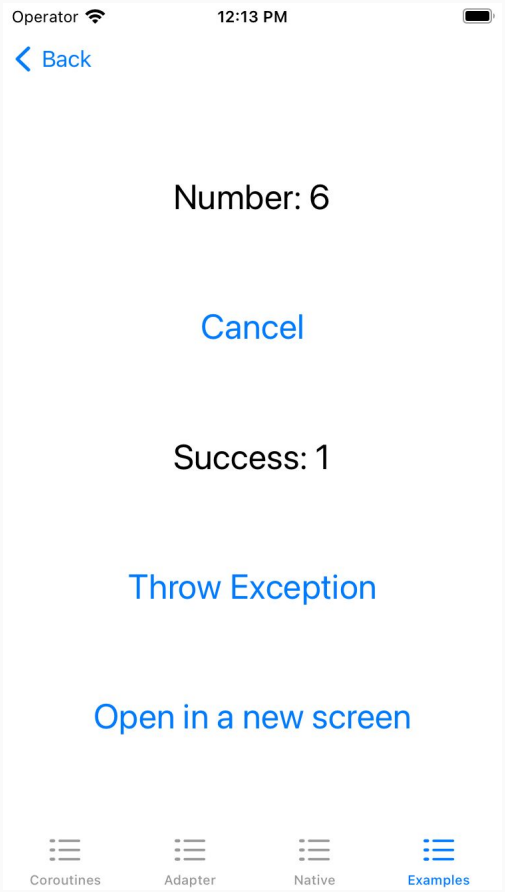
Playground

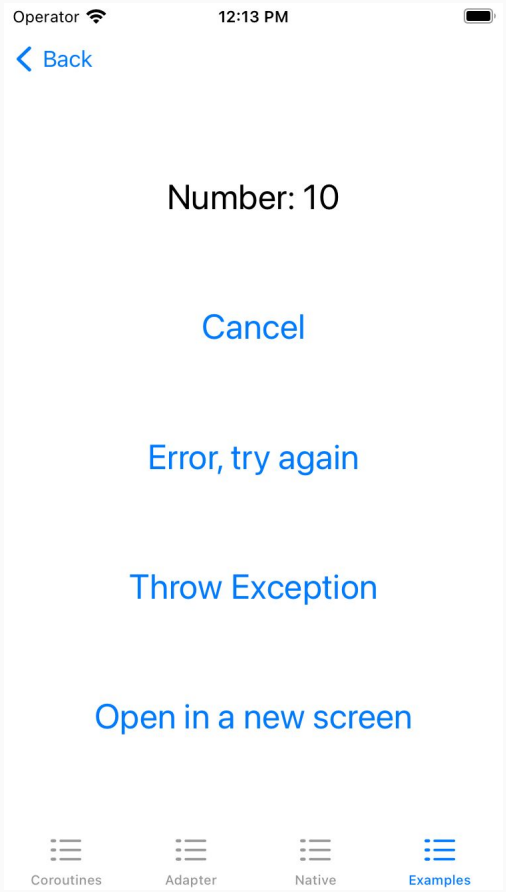

Operator
12:09 PM
Number: 2
Cancel
Generate result
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples


Operator
12:13 PM
Back
Number: 6
Cancel
Success: 1
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples

Playground


Operator
12:09 PM
Number: 2
Cancel
Generate result
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples


Operator
12:13 PM
Back
Number: 6
Cancel
Success: 1
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples


Operator
12:13 PM
Back
Number: 10
Cancel
Error, try again
Throw Exception
Open in a new screen
Coroutines
Adapter
Native
Examples

Published Int

```
@NativeCoroutines
val numberFlow: Flow<Int> = flow {
    var i = 0
    while (true) {
        emit(i++)
        delay(1000)
    }
}
```

Published Int

```
private class CoroutinesExampleModel: ObservableObject {

    @Published
    var number: Int = -1

    func activate() {
        createPublisher(for: viewModel.numberFlow)
            .sink { _ in
            } receiveValue: { [weak self] number in
                self?.number = number.intValue
            }
            .store(in: &cancellables)
    }

}
```

Published Int

```
private class CoroutinesExampleModel: ObservableObject {

  @Published
  var number: Int = -1

}
```

Published Int

```
private class CoroutinesExampleModel: ObservableObject {

  @Published
  var number: Int = -1

  func activate() {
      createPublisher(for: viewModel.numberFlow)




  }
}
```

Published Int

```
private class CoroutinesExampleModel: ObservableObject {

  @Published
  var number: Int = -1

  func activate() {
      createPublisher(for: viewModel.numberFlow)
          .sink { _ in
          } receiveValue: { [weak self] number in
              self?.number = number.intValue
          }
          .store(in: &cancellables)

  }
}
```

Published Int

```
private class CoroutinesExampleModel: ObservableObject {

  @Published
  var number: Int = -1

  func activate() {
      createPublisher(for: viewModel.numberFlow)
          .sink { _ in
          } receiveValue: { [weak self] number in
              self?.number = number.intValue
          }
          .store(in: &cancellables)

  }
}
```

Published Int

```
private class CoroutinesExampleModel: ObservableObject {

  @Published
  var number: Int = -1

  func activate() {
      createPublisher(for: viewModel.numberFlow)
          .sink { _ in
          } receiveValue: { [weak self] number in
              self?.number = number.intValue
          }
          .store(in: &cancellables)
  }
}
```

Published Int

```
struct CoroutinesExampleScreen: View {

    @StateObject
    private var observableModel = CoroutinesExampleModel()

    var body: some View {
        Text("Number: \(observableModel.number)")
    }
}
```

Published Int

```
struct CoroutinesExampleScreen: View {

  @StateObject
  private var observableModel = CoroutinesExampleModel()

  var body: some View {
      Text("Number: \(observableModel.number)")
  }

}
```

Published Int

```
struct CoroutinesExampleScreen: View {

  @StateObject
  private var observableModel = CoroutinesExampleModel()

  var body: some View {
      Text("Number: \(observableModel.number)")
  }
}
```

Published Int

Number: 3

Number: 13

Number: 28

Number: 47

KmmViewModel - Bonus

- The same author
- Generates ObservableObject itd.

KmmViewModel - Bonus

- The same author
- Generates ObservableObject itd.

ObservableObject	KMMViewModel
@StateObject	@StateViewModel
@ObservedObject	@ObservedViewModel
@EnvironmentObject	@EnvironmentViewModel
environmentObject(_:)	environmentViewModel(_:)

Suspend Exception Handling

```
@NativeCoroutines
suspend fun throwException() {
    delay(1000)
    throw IllegalStateException()
}
```

Suspend Exception Handling

```
print("future exception start")

createFuture(for: viewModel.throwException())
    .sink { completion in
        print("future exception completion \(completion)")
    } receiveValue: { value in
    }
```

Suspend Exception Handling

```
print("future exception start")

createFuture(for: viewModel.throwException())
    .sink { completion in
        print("future exception completion \(completion)")
    } receiveValue: { value in
    }
```

```
18:07:03 future exception start
18:07:04 future exception completion failure(Error Domain=KotlinExcepton
Code=0 "(null)" UserInfo={KotlinException=kotlin.IllegalStateException})
```

Flow Exception Handling

```
@NativeCoroutines
val errorFlow: Flow<Int> = flow {
    repeat(3) { number ->
        emit(number)
        delay(1000)
    }
    throw IllegalStateException()
}
```

Flow Exception Handling

```
createPublisher(for: viewModel.errorFlow)
    .sink { completion in
        print("publisher exception completion \(completion)")
    } receiveValue: { number in
        print("publisher exception recieveValue \(number)")
    }
```

Flow Exception Handling

```
createPublisher(for: viewModel.errorFlow)
    .sink { completion in
        print("publisher exception completion \(completion)")
    } receiveValue: { number in
        print("publisher exception recieveValue \(number)")
    }
```

```
2023-03-21 18:14:24 publisher exception recieveValue 0
2023-03-21 18:14:25 publisher exception recieveValue 1
2023-03-21 18:14:26 publisher exception recieveValue 2
2023-03-21 18:14:27 publisher exception completion
failure(Error Domain=KotlinException Code=0 "(null)"
UserInfo={KotlinException=kotlin.IllegalStateException})
```

Exception Handling

- Exception is a Completion in Combine
- Exception handling is hard in Swift
- Better to return an error value

```
@NativeCoroutines
suspend fun throwException() {
    delay(1000)
    throw IllegalStateException()
}
```

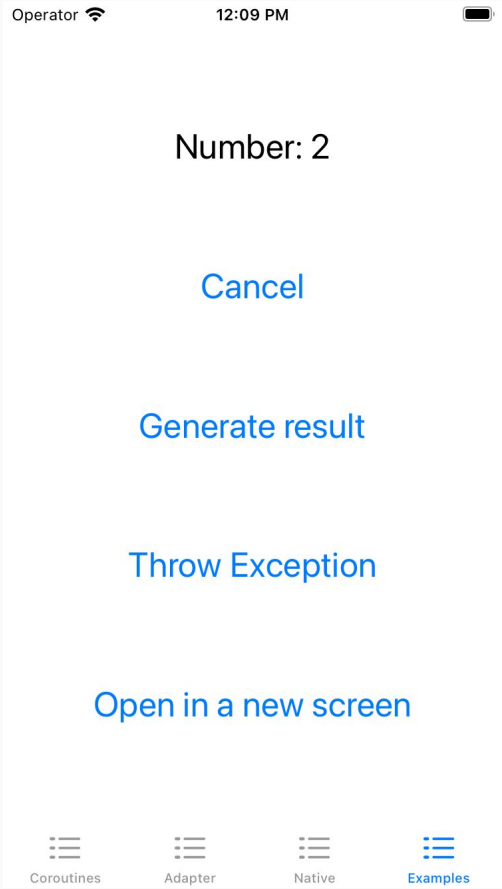
```
@NativeCoroutines
val errorFlow: Flow<Int> = flow {
    repeat(3) { number ->
        emit(number)
        delay(1000)
    }
    throw IllegalStateException()
}
```

Cancellation

```
@NativeCoroutines
val numberFlow: Flow<Int> = flow {
    var i = 0
    while (true) {
        emit(i++)
        delay(1000)
    }
}.onEach { number ->
    log.i("numberFlow onEach: $number")
}.onCompletion {throwable ->
    log.i("numberFlow onCompletion: $throwable")
}
```

Manual cancellation

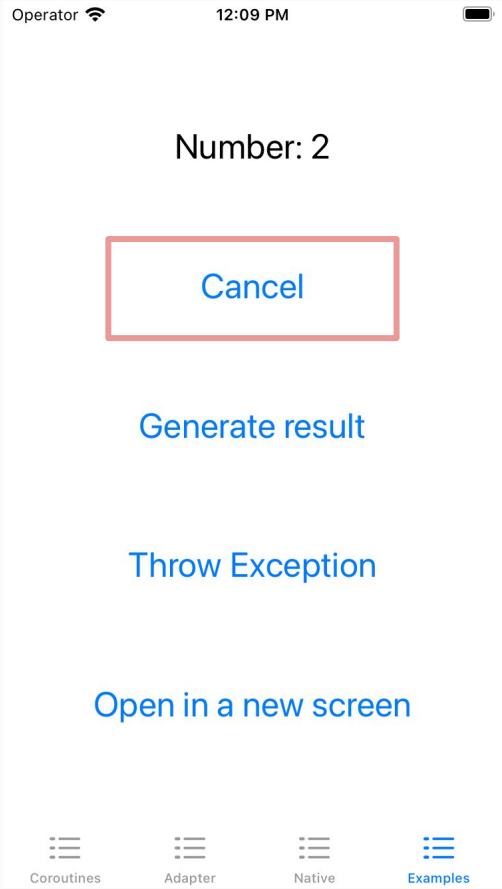
```
createPublisher(
  for: viewModel.numberFlow
)
  .sink(...)
  .store(in: &cancellables)
```



Manual cancellation

```
createPublisher(
  for: viewModel.numberFlow
)
  .sink(...)
  .store(in: &cancellables)

func cancel() {
    cancellables.forEach { $0.cancel() }
    cancellables.removeAll()
}
```

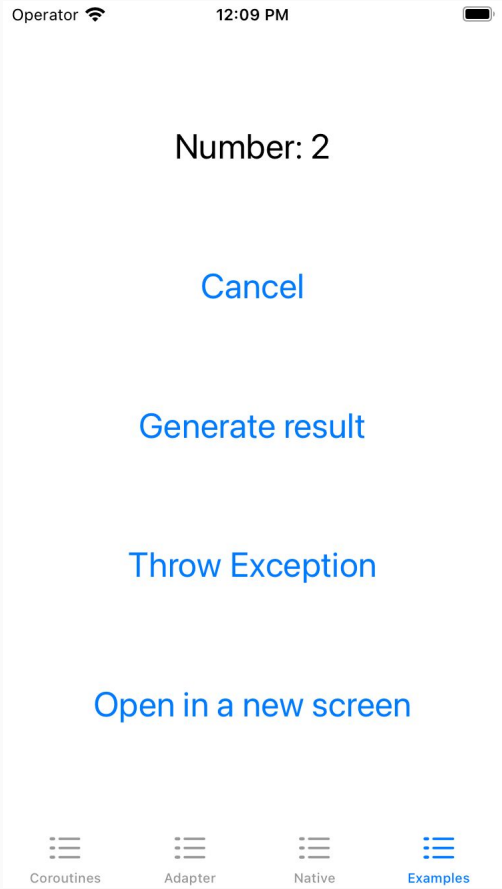


Manual cancellation

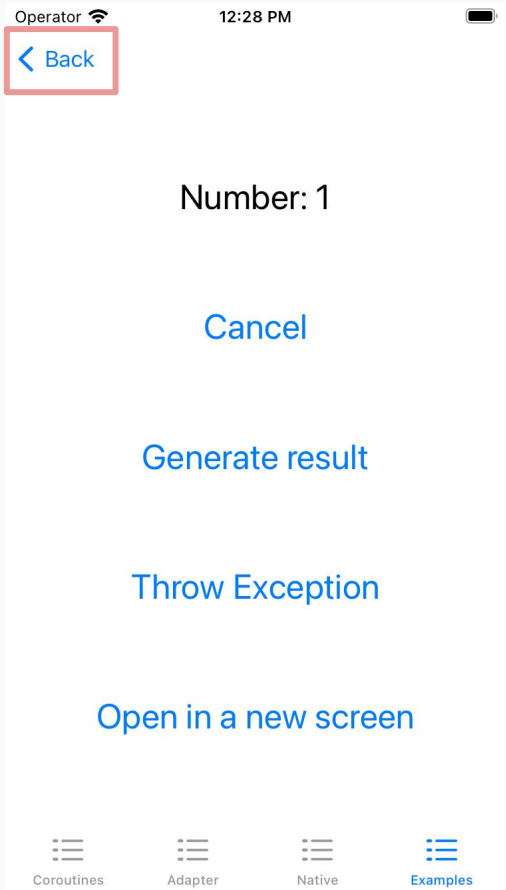
```
createPublisher(
  for: viewModel.numberFlow
)
  .sink(...)
  .store(in: &cancellables)

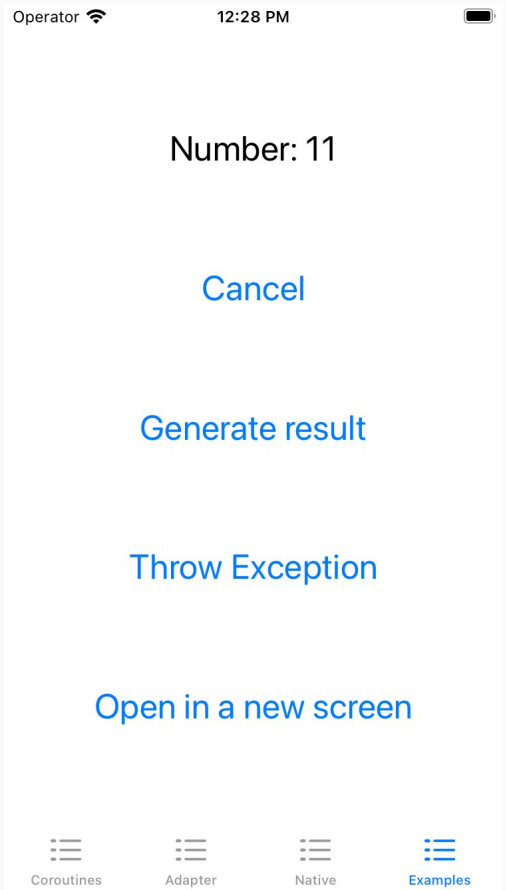

func cancel() {
    cancellables.forEach { $0.cancel() }
    cancellables.removeAll()
}
```

```
2023-03-24 12:21:30 numberFlow onEach: 0
2023-03-24 12:21:31 numberFlow onEach: 1
2023-03-24 12:21:32 numberFlow onEach: 2
2023-03-24 12:21:33 numberFlow onCompletion:
kotlinx.coroutines.JobCancellationException: …
```



Automatic cancellation





Automatic cancellation

```
createPublisher(
  for: viewModel.numberFlow
)
  .sink(...)
  .store(in: &cancellables)
```

```
2023-03-24 12:28:13 init 0x44af1d53beeb
2023-03-24 12:28:13 numberFlow onEach: 0
2023-03-24 12:28:14 numberFlow onEach: 1
2023-03-24 12:28:15 numberFlow onEach: 2
2023-03-24 12:28:16 numberFlow onEach: 3
2023-03-24 12:28:16 deinit 0x44af1d53beeb
2023-03-24 12:28:16 numberFlow onCompletion:

kotlinx.coroutines.JobCancellationException: ...
```

Cancellation

- Freeing up resources
- Avoiding unnecessary networking
- Avoiding Memory leaks

```
var cancellables = [AnyCancellable]()

...
    .store(in: &cancellables)


func cancel() {
    cancellables.forEach { $0.cancel() }
    cancellables.removeAll()
}
```

Summary

Summary

- Different ways of Suspend and Flow from swift
- Integrating KMP-NativeCoroutines with SwiftUi
- Exception Handling
- Cancellation

Learning Kotlin Multiplatform

- https://johnoreilly.dev/
- https://touchlab.co/blog/
- https://www.marcogomiero.com/
- https://akjaw.com/

Sources

- https://github.com/AKJAW/Swift-Coroutines-With-KMP-NativeCoroutines
- https://github.com/touchlab/KaMPKit
- https://github.com/rickclephas/KMP-NativeCoroutines
- https://github.com/rickclephas/KMM-ViewModel
- https://dev.to/touchlab/kotlin-coroutines-and-swift-revisited-j5h
- https://www.slideshare.net/ChristianMelchior/coroutines-for-kotlin-multiplatform-in-practise

Questions?